



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Aprendizaje Automático Probabilístico

Redes neuronales para secuencias (actividades)

Alfons Juan

DSIC

Departamento de Sistemas
Informáticos y Computación

Índice

15.1	Introducción	1
15.2	Redes neuronales recurrentes	2
15.2.1	Vec2Seq: generación de secuencias	2
15.2.2	Seq2Vec: clasificación de secuencias	3
15.2.3	Seq2Seq: traducción de secuencias	4
15.2.4	Aprendizaje forzado	5
15.2.5	Backpropagation through time	6
15.2.6	Gradientes que desaparecen y explotan	7
15.2.7	Gating y long term memory	8
15.2.8	Beam search	9
15.3	CNNs unidimensionales	10
15.4	Atención	11
15.4.1	Atención: búsqueda suave en diccionario	11
15.4.2	Regresión kernel: atención no paramétrica	12
15.4.3	Atención paramétrica	13
15.4.4	Seq2Seq con atención	14
15.4.6	Seq+Seq2Vec con atención	15

15.5 Transformers	16
15.5.1 Auto-atención	17
15.5.2 Atención multi-cabecal	18
15.5.3 Codificación posicional	19
15.5.4 Todo junto	20
15.5.5 Comparación de transformers, CNNs y RNNs	21
15.7 Modelos de lenguaje	22
15.7.1 ELMo	23
15.7.2 BERT	24
15.7.3 GPT	25
15.7.4 T5	26
15.7.5 Discusión	27

15.1. Introducció

- Las redes neuronales para secuencias obtienen resultados punteros en muchas aplicaciones clásicas del aprendizaje automático. Indica cuál de las siguientes aplicaciones no se halla entre ellas.

1. Reconocimiento automático del habla

2. Traducción automática

3. Clasificación de texto

4. Todas se hallan entre ellas

15.2. Redes neuronales recurrentes

15.2.1. Vec2Seq: generación de secuencias

- Reproduce el cuaderno de la [Figura 15.2](#) para implementar una RNN de modelado de lenguaje (a nivel de carácter) entrenada con *The Time Machine* de H. G. Wells.

1 the githa some thong the time traveller held in his hand was a
2 glitteringmetallic framework scarcely larger than a small clock and
3 verydelicately made there was ivory in it and the latter than s bettyre
4 tat howhong s ie time thave ler simk you a dimensions le ghat dionthat
5 shall travel indifferently in any direction of space and timeas the
6 driver determinesfilby contented himself with laughterbut i have
7 experimental verification said the time travellerit would be remarkably
8 convenient for the histo

15.2.2. Seq2Vec: clasificación de secuencias

- ▶ Reproduce el cuaderno [rnn_sentiment_jax.ipynb](#) para implementar una RNN bidireccional de análisis de sentimientos entrenada con IMDB movie review dataset.

▷ Prueba la RNN en la sección Testing con alguna frase:

```
1 predict_sentiment(net, vocab, "this movie is really good")
```

```
1 'positive'
```

15.2.3. Seq2Seq: traducción de secuencias

- ▶ Reproduce el cuaderno [nmt_torch.ipynb](#) para implementar una RNN con arquitectura encoder-decoder de NMT Francés→Inglés entrenada con datos del [proyecto Tatoeba](#).
- ▷ Estudia la métrica BLEU (Bilingual Evaluation Understudy) en la sección Evaluation y prueba algún par de frases:

```
1 engs = ["I'm calm ."]  
2 fras = ["je suis calme ."]
```

	English	Truth	Prediction	Bleu
1				
2	0	I'm calm .	je suis calme .	1.0

15.2.4. Aprendizaje forzado

- En entrenamiento de un modelo de lenguaje, la verosimilitud de una secuencia de palabras $w_1, w_2, \dots, w_T$ es

$$p(\mathbf{w}_{1:T}) = \prod_{t=1}^T p(w_t \mid \mathbf{w}_{1:t-1})$$

Aprendizaje forzado con una RNN fija la entrada a $x_t = w_{t-1}$ y la salida a $y_t = w_t$, tomando como etiquetas del pasado, $\mathbf{w}_{1:t-1}$:

1. La ground truth, esto es, las etiquetas de referencia
2. Las etiquetas generadas por el modelo
3. Tanto las de referencia como las generadas, decidiendo al azar, pero empezando por las de referencia e incrementando gradualmente la probabilidad de usar las generadas
4. Ninguna de las anteriores

15.2.5. Backpropagation through time

- Backpropagation through time (BPTT) es backprop aplicado al grafo computacional desenrollado en el tiempo. Consulta la sección [Analysis of Gradients in RNNs de d2l.ai](#) y prueba que:

$$a_t = b_t + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^t c_j \right) b_i \quad \text{para } t \geq 1$$

donde $\{a_t\}$, $\{b_t\}$ y $\{c_t\}$ son tres secuencias tales que $a_0 = 0$ y $a_t = b_t + c_t a_{t-1}$. Así a_t se calcula sin recurrir a ningún $a_{t'}$ previo.

15.2.6. Gradientes que desaparecen y explotan

- ▶ Las activaciones en una RNN tienden a desaparecer o explotar tanto en forward, ya que multiplicamos la matriz $\mathbf{W}_{hh}$ en cada t , como en backward, ya que multiplicamos Jacobianas en cada t . Una solución sencilla para los que explotan es gradient clipping. En relación con los que desaparecen, indica cuál de las siguientes propuestas es una solución bien conocida y estudiada.
 1. Controlar el radio espectral λ del mapping forward, $\mathbf{W}_{hh}$, y del backward, dado por la Jacobiana $\mathbf{J}_{hh}$
 2. Optimizar $\mathbf{W}_{hh}$ con restricciones, manteniéndola ortogonal
 3. Modificar la arquitectura de la RNN con actualizaciones aditivas de los estados ocultos
 - 4.** Todas las anteriores

15.2.7. Gating y long term memory

- ▶ Las “vanilla” RNNs son difíciles de entrenar por los gradientes que desaparecen. La solución más extendida consiste en actualizar los estados ocultos de manera aditiva. En particular, se usan celdas de memoria más o menos complejas entre las que destacan la Gated Recurrent Unit (GRU) y la Long Short Term Memory (LSTM). En relación con ellas, indica la respuesta incorrecta.
 1. La LSTM se propuso en 1997, pero no alcanzó gran popularidad hasta unos 20 años más tarde
 2. La GRU se propuso en 2014 y puede verse como una versión simplificada de la LSTM
 - 3.** Ambas se definen en términos de tres puertas (gates)
 4. Todas son correctas

15.2.8. Beam search

- Dada una RNN, la búsqueda exhaustiva de una secuencia MAP,

$$\mathbf{y}_{1:T}^* = \arg \max_{\mathbf{y}_{1:T}} p(\mathbf{y}_{1:T} \mid \mathbf{x})$$

tiene coste $O(V^T)$ con un vocabulario de V palabras. Greedy decoding obtiene una aproximación de coste $O(VT)$ mediante

$$\hat{y}_t = \arg \max_y p(y_t = y \mid \hat{\mathbf{y}}_{1:t}, \mathbf{x}) \quad \text{para } t = 1, 2, \dots$$

Beam search generaliza greedy decoding manteniendo las $K \geq 1$ mejores secuencias candidatas en cada t . Su coste es:

1. $O(KVT)$

2. $O(K^2VT)$

3. $O(KV^2T)$

4. $O(K^2V^2T)$

15.3. CNNs unidimensionales

- ▶ Reproduce el cuaderno [cnn1d_sentiment_jax.ipynb](#) para implementar una CNN 1d de análisis de sentimientos entrenada con IMDB movie review dataset.
- ▷ Prueba la CNN 1d en la sección Testing con alguna frase:

```
1 predict_sentiment(net, vocab, "this movie is really good")
```

```
1 'positive'
```

15.4. Atención

15.4.1. Atención: búsqueda suave en diccionario

- ▶ El mecanismo de atención ganó gran popularidad en NMT tras su introducción en 2014. Indica la respuesta incorrecta.
- 1. NMT era por entonces una aproximación emergente que rivalizaba con la dominante, la aproximación basada en reglas
- 2. Los modelos NMT usuales eran RNNs con arquitectura encoder-decoder que codificaban la frase de entrada en un único vector contextual de longitud fija, lo que parecía un cuello de botella
- 3. Permite que el modelo pueda buscar suavemente partes de la entrada relevantes en la predicción de la palabra de salida
- 4. Calcula un promedio ponderado de todos los estados ocultos de la secuencia de entrada para construir el vector contextual

Bahdanau et al., 2014.

15.4.2. Regresión kernel: atención no paramétrica

- Atención se entiende fácilmente con el modelo de regresión Nardaya-Watson (NW) propuesto en 1964. Dados n pares de entrenamiento $\{(x_i, y_i) \in \mathbb{R}^2\}$, queremos predecir la salida $\hat{y}$ para una query (muestra) de test x . Indica la respuesta incorrecta.

1. Un predictor trivial es una media simple, $\hat{y} = \frac{1}{n} \sum_i y_i$
2. NW propone usar una media ponderada, $\hat{y} = \sum_i \alpha(x, x_i) y_i$, donde $\alpha(x, x_i)$ mide la relevancia de x_i para predecir x
3. Una elección usual para la función de ponderación (atención) $\alpha(x, x_i)$ es un kernel Gaussiano normalizado
4. Todas son correctas

Chaudhari et al., 2021.

15.4.3. Atención paramétrica

- ▶ Reproduce el cuaderno [attention_jax.ipynb](#) con código de ejemplo para implementar atención suave.

15.4.4. Seq2Seq con atención

- ▶ Reproduce el cuaderno [nmt_attention_jax.ipynb](#) para implementar NMT usando una RNN con decoder basado en atención.
- ▷ Prueba algún par de frases en la sección Evaluation:

```
1 engs = ["I'm calm ."]  
2 fras = ["je suis calme ."]
```

```
1 I'm calm . => je suis calme .,  bleu 1.000
```

15.4.6. Seq+Seq2Vec con atención

- ▶ Reproduce el cuaderno [entailment_attention_mlp_jax.ipynb](#) para implementar textual entailment (inferencia en lenguaje natural) usando un MLP con atención.

▷ Prueba algunos pares de frases en la sección Evaluation:

```
1 predict_snli(state, vocab, ["he", "is", "short", "."], ["he",  
  ↪  "is", "tall", "."])
```

```
1 'contradiction'
```

```
1 predict_snli(state, vocab, ["he", "is", "a", "student", "."],  
  ↪  ["he", "is", "tall", "."])
```

```
1 'neutral'
```

```
1 predict_snli(state, vocab, ["he", "is", "studying", "."],  
  ↪  ["he", "is", "learning", "."])
```

```
1 'entailment'
```

15.5. Transformers

- ▶ Transformer es un modelo seq2seq de gran popularidad tras su introducción en 2017. Indica la respuesta incorrecta.
 1. Aunque se introdujo como modelo de NMT, su uso se ha extendido a otras tareas, no solo de NLP
 2. Los modelos usuales de NMT en 2017 eran RNNs con arquitectura encoder-decoder y decoder basado en atención
 3. Transformer propuso emplear atención tanto en el encoder como en el decoder, eliminando así la necesidad de RNNs
 4. Todas son correctas

Vaswani et al., 2017.

15.5.1. Auto-atención

- Dada una secuencia de entrada $\{\mathbf{x}_i \in \mathbb{R}^d\}_{i=1}^n$, auto-atención en el encoder genera una secuencia de salida de la misma longitud,

$$\mathbf{y}_i = \text{Attn}(\mathbf{x}_i, (\mathbf{x}_1, \mathbf{x}_1), \dots, (\mathbf{x}_n, \mathbf{x}_n))$$

Para el decoder usamos una expresión parecida:

1. $\mathbf{y}_i = \text{Attn}(\mathbf{y}_{i-1}, (\mathbf{y}_1, \mathbf{y}_1), \dots, (\mathbf{y}_n, \mathbf{y}_n))$

2. $\mathbf{y}_i = \text{Attn}(\mathbf{y}_{i-1}, (\mathbf{y}_1, \mathbf{y}_1), \dots, (\mathbf{y}_{i-1}, \mathbf{y}_{i-1}))$

3. $\mathbf{y}_i = \text{Attn}(\mathbf{y}_{i-1}, (\mathbf{y}_1, \mathbf{y}_1), \dots, (\mathbf{y}_i, \mathbf{y}_i))$

4. Ninguna de las anteriores

15.5.2. Atención multi-cabezal

- ▶ Reproduce el cuaderno de la [multi_head_attention_jax.ipynb](#) con código de ejemplo para implementar multi-head attention (MHA).

15.5.3. Codificación posicional

- ▶ En el encoder de transformer, la codificación de la posición de una palabra se lleva a cabo mediante un entero:
 1. Tal cual
 2. En formato binario
 - 3.** En términos de funciones base sinusoidales
 4. Ninguna de las anteriores

15.5.4. Todo junto

- ▶ Reproduce el cuaderno [transformers_jax.ipynb](#) para implementar NMT con transformers.
- ▷ Prueba algún par de frases en la sección Evaluation:

```
1 engs = ["I'm calm ."]  
2 fras = ["je suis calme ."]
```

```
1 I'm calm . => je suis calme .,  bleu 0.783
```

15.5.5. Comparación de transformers, CNNs y RNNs

- Un factor clave en la comparación de auto-atención (transformers), CNNs y RNNs es el número de operaciones secuenciales que deben realizarse en función de la longitud de la secuencia, n . Indica la respuesta correcta.

1. $O(1)$ auto-atención y RNNs, $O(n)$ CNNs

2. $O(1)$ auto-atención y CNNs, $O(n)$ RNNs

3. $O(1)$ CNNs y RNNs, $O(n)$ auto-atención

4. Ninguna de las anteriores

15.7. Modelos de lenguaje y aprendizaje de representación no supervisado

- ▶ A finales de los 2010, el uso de word embeddings incontextuales se ha extendido al de contextuales. En relación con las representaciones contextuales, indica la respuesta incorrecta.
 1. Se suelen derivar de estados latentes de RNNs o transformers auto-regresivos (solo-decoder) para modelado de lenguaje
 2. Las representaciones producidas por modelos ya pre-entrenados (con muchos datos) se suelen ajustar (fine-tuning) a una tarea específica con datos propios de la misma
 - 3.** Es usual distinguir entre modelos causales y no-causales; los primeros no generan texto, mientras que los segundos sí
 4. Todas son correctas

15.7.1. ELMo

- ▶ Embeddings from Language Model (ELMo), propuesto en 2018, es un modelo pionero de representación contextual de palabras. Indica la respuesta incorrecta.
 1. Ajusta dos modelos de lenguaje RNN, uno izqda-dcha y otro dcha-izqda, y combina sus estados ocultos para formar un embedding por palabra
 2. En fine-tuning se aprende un conjunto lineal de pesos dependiente de la tarea para transformar el embedding ELMo de cada palabra en su embedding final
 - 3.** Se ha aplicado con buenos resultados en WSD (desambiguación lingüística) y POS tagging (etiquetado gramatical)
 4. Todas son correctas

Clark et al., 2018.

15.7.2. BERT

- ▶ Bidirectional Encoder Representations from Transformers (BERT), propuesto en 2019, es un transformer para transformar una versión modificada de una secuencia de vuelta a su versión no modificada. Indica la respuesta incorrecta.
1. La tarea que inspira el objetivo de pre-entrenamiento de BERT se conoce como tarea fill-in-the-blank o cloze
 2. A diferencia de ELMo, el ajuste (fine-tuning) de BERT se limita a añadir uno o más cabezales de salida a la capa oculta final para adaptarlo a la tarea deseada
 3. Se ha aplicado con éxito en diversas tareas NLP, tanto a nivel de palabra (p.e. POS tagging) como a nivel de frase (p.e. clasificación de frases o pares de frases)
 4. Todas son correctas

Devlin et al., 2019 [el repo oficial.](#)

15.7.3. GPT

- ▶ Generative Pre-training Transformer (GPT), propuesto en 2018, es un masked transformer decoder. Indica la respuesta incorrecta.
 1. A diferencia de ELMo y BERT, es un modelo causal
 2. Se entrenó conjuntamente para modelado de lenguaje (con muchos datos no supervisados) y clasificación en tareas específicas (con pocos datos supervisados)
 3. Sus sucesores GPT-2 (2019) y GPT-3 (2020) se han entrenado para modelado de lenguaje únicamente
 4. Todas son correctas

15.7.4. T5

- ▶ Text-to-text Transfer Transformer (T5), propuesto en 2019, es un masked transformer seq2seq2. Indica la respuesta incorrecta.
 1. Se propuso para explorar hasta qué punto un único modelo puede resolver múltiples tareas
 2. Se pre-entrenó no supervisadamente como modelo de lenguaje enmascarado
 3. Tras el pre-entrenamiento no supervisado, se ajustó supervisadamente para varias tareas específicas
 4. Todas son correctas